WAZUH DETECTION LAB

Custom Decoders, Correlation Rules & MITRE ATT&CK Mapping



Built with Wazuh • RBAC • Dashboards

Executive Summary

When Three Wrong Passwords and One Right One Tell a Story

Every application leaves tiny footprints in a log file. One failed login is just a typo. Three failed logins followed by a success looks like someone patiently trying keys until a lock gives way. A Security Operations Center (SOC) analyst cannot read every line by eye, so the system must read the story for them.

This lab teaches Wazuh to do exactly that. Plain text from a sample application is cut into clear fields and then climbs a ladder of alerts. A quiet failed login stays small. A lock-picking pattern becomes a high alert. A sudden jump to administrator power is flagged with its matching MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) labels.

The story ends with a careful hand-over. A new analyst account can look at the alerts but cannot change a thing, and one dashboard turns scattered lines into a picture that can be understood in seconds.

Table of Contents

Objective **2**
Part A: Building the Detection Rules **2**
Telling Wazuh Which Log File to Read 2
Writing the Decoders 3
Writing the Base Rules 4
Writing the Correlation Rule 5
Fixing the Rule Loading Error 6
Upgrading the Privilege Escalation Rule 7
Part B: Proving the Rules with wazuh-logtest **7**
Starting wazuh-logtest and Reading Its Phases 7
Testing a Failed Login 8
Testing Brute Force Followed by Success 9
Testing Privilege Escalation 9
Part C: Role Based Access Control for the SOC Analyst **10**
Creating the Analyst Role 10
Setting Cluster, Index and Tenant Permissions 11
Creating the Analyst User and Mapping the Role 13
Part D: Verifying Analyst Access and Alerts **15**
Logging In as the Analyst 15
Checking the Access Limit After Login 16
Searching the Alerts 16
Part E: Dashboard Design **18**
Saving a High Severity Search 18
Building the Visualization 19
Creating the Dashboard 21
Conclusion **22**

Objective

- ➤ Build a detection pipeline in **Wazuh** for the log file **`/var/log/myapp.log`** on the Wazuh server Virtual Machine (VM).
- ➤ Write **decoders** and **rules** that turn raw log text into alerts for failed login, successful login and privilege escalation.
- ➤ Write a **correlation rule** (level 12) for failed logins followed by a success.
- ➤ Raise the privilege escalation rule to level 13 with **MITRE ATT&CK** ids T1548 and T1078 and prove every rule with **`wazuh-logtest.`**
- ➤ Create a read-only analyst role with **Role Based Access Control (RBAC)** and show the alerts in a saved search, a visualization and a dashboard.

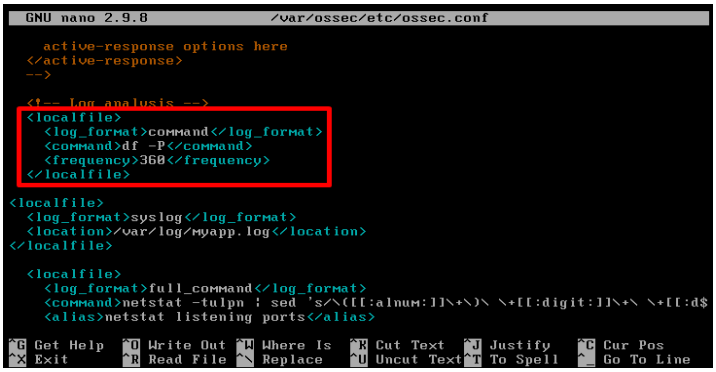
Part A: Building the Detection Rules

Telling Wazuh Which Log File to Read

⚙ **Purpose:** Wazuh reads only files it is told about. A localfile block adds the log file.

⌨ **Command:** **`sudo nano /var/ossec/etc/ossec.conf`**

- ▫ **`sudo`:** runs the command with administrator rights.
- ▫ **`nano`:** a simple text editor that works inside the terminal.
- ▫ **`/var/ossec/etc/ossec.conf`:** the main configuration file of the Wazuh manager.



Screenshot: ossec.conf opened in nano with the myapp.log entry

- ▫ **Expected output:** the file opens in nano and the localfile entry for **`/var/log/myapp.log`** is visible.
- ▫ **`<log_format>syslog</log_format>`** tells Wazuh to read the file line by line as normal log text.
- ▫ **`<location>/var/log/myapp.log</location>`** is the full path of the file that Wazuh watches.

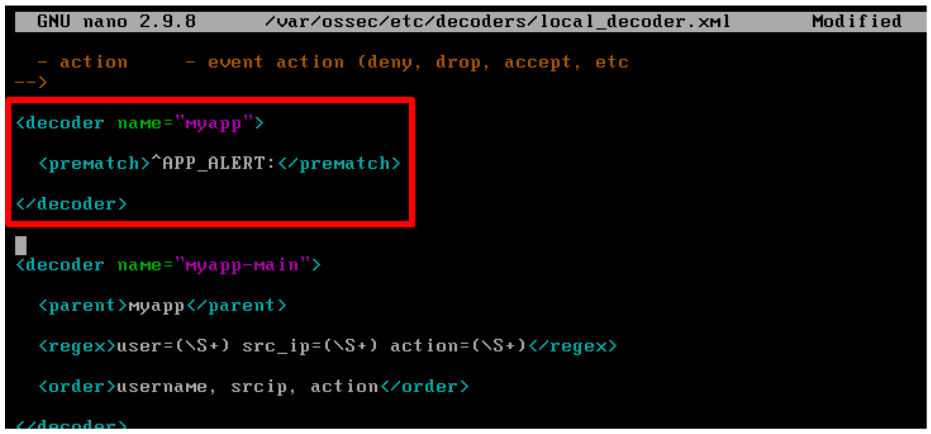
❖ **Why it matters:** without this block, no line reaches the decoders or rules. Wazuh loads the file at start, so **restart** the manager after saving.

Writing the Decoders

⚙ **Purpose:** a **decoder** cuts one raw log line into named fields such as user, source Internet Protocol (IP) address and action. Rules can only check what a decoder has already extracted. The file uses Extensible Markup Language (XML).

⌨ **Command: `sudo nano /var/ossec/etc/decoders/local_decoder.xml`**

- **`/var/ossec/etc/decoders/local_decoder.xml`:** the file for custom decoders, so Wazuh updates do not erase them.



Screenshot: parent decoder myapp and child decoder myapp-main



Screenshot: child decoder myapp-cmd

- **Expected output:** one parent decoder and two child decoders are visible.
- **Parent `myapp`: `<prematch>^APP_ALERT:</prematch>`** accepts only lines that start with **`APP_ALERT:`** The sign ^ means the start of the line.
- **Child `myapp-main`: `<parent>myapp</parent>`** links it to the parent. The expression **`user=(\S+) src_ip=(\S+) action=(\S+)`** captures three values. `\S+` means one or more characters that are not spaces.
- **`<order>` tag** names the captured values in the same order: username, srcip, action.
- **Child `myapp-cmd`:** cmd="(\.+)" keeps the text inside the quotes after `cmd=`. In Wazuh, `\.+` means one or more of any character.

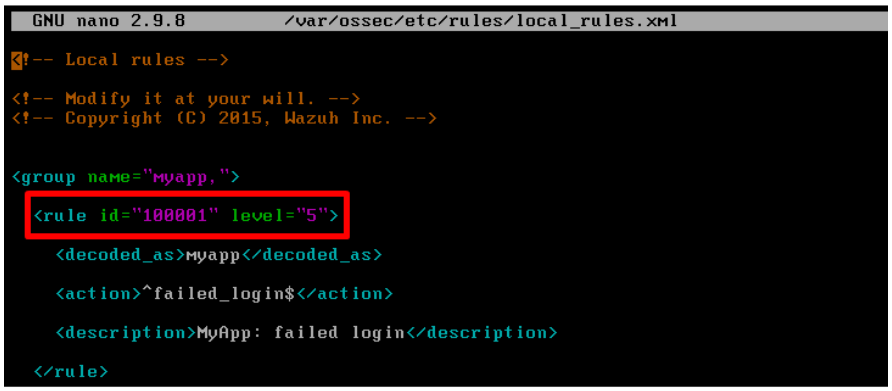
❖ **Result:** each APP_ALERT line becomes fields that later appear as data.username, data.srcip, data.action and data.cmd.

Writing the Base Rules

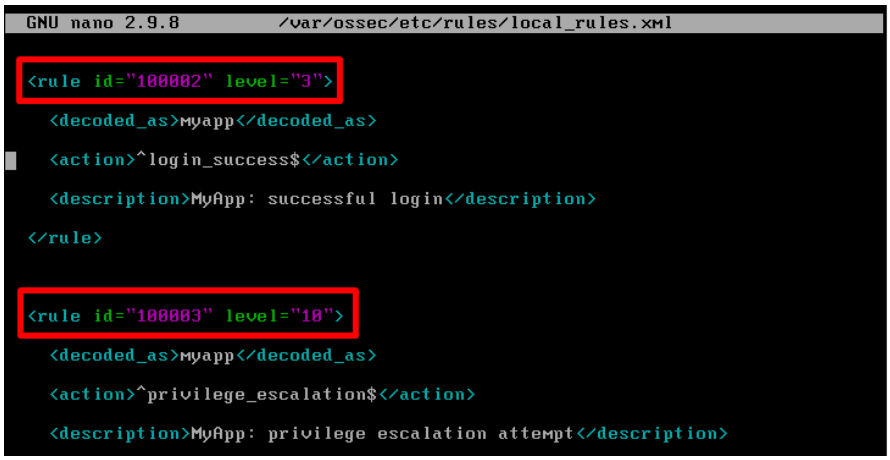
⚙ **Purpose:** a base rule watches one kind of event. Three small rules give each event its own id and level. Custom rule ids start at 100000, so they never clash with the rules that ship with Wazuh.

⌨ **Command: `sudo nano /var/ossec/etc/rules/local_rules.xml`**

- **`/var/ossec/etc/rules/local_rules.xml`:** the file for custom rules. sudo and nano mean the same as before.



Screenshot: rule 100001 for failed login



Screenshot: rules 100002 and 100003

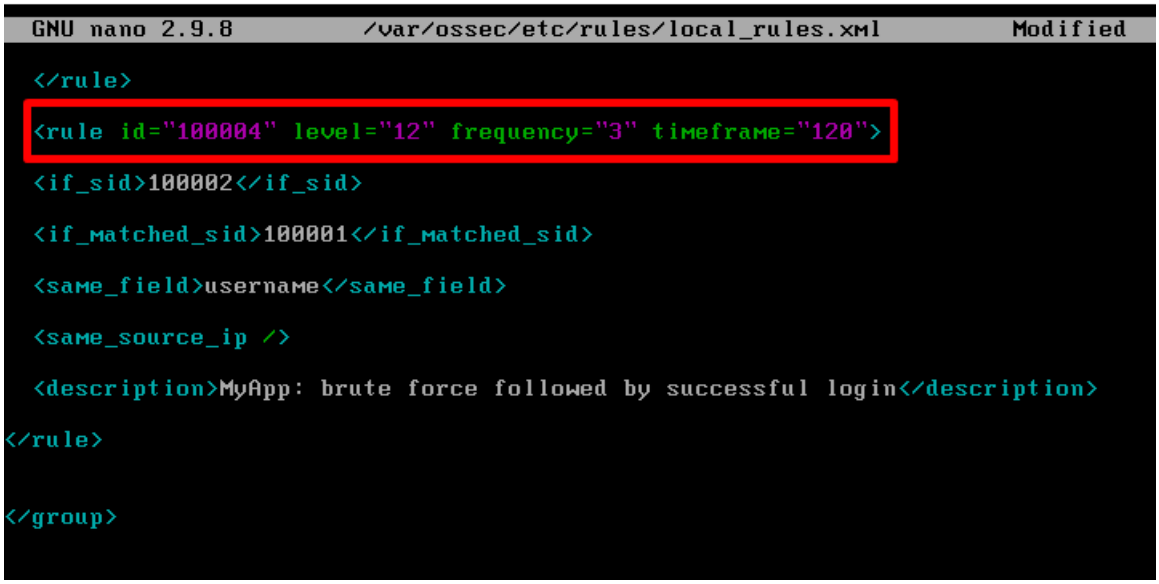
- **Expected output:** three rules inside the group myapp.
- **`<decoded_as>myapp</decoded_as>`** makes the rule check only events that the myapp decoder handled.
- **`<action>^failed_login$</action>`** matches the decoded action exactly. `^` is the start and `$` is the end of the value.
- **Rule 100001:** level 5, failed login. **Rule 100002:** level 3, successful login. **Rule 100003:** level 10 at this point, privilege escalation attempt.

❖ **Level meaning:** levels run from 0 to 15. A higher number means a more serious event.

Writing the Correlation Rule

⚙ **Purpose:** a **correlation rule** joins several events into one story. One failed login is small. Failed logins followed by a success can mean that a password was guessed.

⌨ **Command:** `sudo nano /var/ossec/etc/rules/local_rules.xml` (the same file as the base rules).



Screenshot: correlation rule 100004

- **Expected output:** the new rule 100004 appears before the closing **`</group>`** tag.
- level="12": a high alert.
- frequency="3" and timeframe="120": the earlier rule must match 3 times within 120 seconds.
- **`<if_sid>100002</if_sid>`:** the rule is checked only when a successful login (rule 100002) happens.
- **`<if_matched_sid>100001</if_matched_sid>`:** rule 100001 (failed login) must have matched earlier, inside the timeframe.
- **`<same_field>username</same_field>` and `<same_source_ip />`:** the events must share the same user name and the same IP address. This stops the logs of different people from being mixed.
- **`<description>`:** the text an analyst reads in the alert. A **brute force** attack means trying many passwords repeatedly.

❖ **SOC view:** this pattern often means that the attacker finally guessed the password, so it needs fast attention.

Fixing the Rule Loading Error

⚠ **Purpose:** after saving rules, the manager must be restarted. If a rule has a mistake, the manager refuses to start. A test command shows the exact reason.

⌨ **Command 1: `sudo systemctl restart wazuh-manager`**

- **`systemctl`:** controls system services.
- **`restart`:** stops the service and starts it again.
- **`wazuh-manager`:** the name of the Wazuh manager service.

⌨ **Command 2: `sudo /var/ossec/bin/wazuh-analysisd -t`**

- **`/var/ossec/bin/wazuh-analysisd`:** the Wazuh analysis engine that loads decoders and rules.
- **`-t`:** test mode. It only checks the configuration and does not start the engine.

```
etails.
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager
Job for wazuh-manager.service failed because the control process exited with err
or code. See "systemctl status wazuh-manager.service" and "journalctl -xe" for d
etails.
[wazuh-user@wazuh-server ~]$ sudo /var/ossec/bin/wazuh-analysisd -t
2026/10/20 09:33:20 wazuh-analysisd: ERROR: Failure to read rule 100001. Field '
action' is static.
2026/10/20 09:33:20 wazuh-analysisd: CRITICAL: (1220): Error loading the rules:
'etc/rules/local_rules.xml'.
[wazuh-user@wazuh-server ~]$
```

Screenshot: rule loading error

⚠ **Problem:** the restart failed. The test printed that rule 100001 could not be read because the field **`action`** is static, followed by error 1220, which says the rules could not be loaded.

- **Reason:** `action` is a **static field** in Wazuh. Static fields have their own tag, so the rule must use the **`<action>`** tag and not a general field check.
- **Fix:** use **`<action>^failed_login$</action>`** as shown in the final rules, run the test again, then restart the manager.
- **Hint in the output:** the system suggests **`systemctl status wazuh-manager.service`** and **`journalctl -xe`**. **`status`** shows the current state of the service. `journalctl` reads the system log, `-x` adds extra explanation and `-e` jumps to the end. Command 2 gives the clearer reason here.

❖ **Good habit:** always run the test before a restart. A broken rules file stops the manager, and then nothing is monitored.

Upgrading the Privilege Escalation Rule

⚙ **Purpose:** privilege escalation is more dangerous than a failed login. Rule 100003 is raised to level 13 and tagged with MITRE ATT&CK techniques.

⌨ **Command:** `sudo nano /var/ossec/etc/rules/local_rules.xml`



Screenshot: rule 100003 upgraded with MITRE ids

- **Expected output:** rule 100003 shows level 13 and a **`<mitre>`** block with two ids.
- **`T1548`** is Abuse Elevation Control Mechanism. It means misusing tools such as sudo to gain higher rights.
- **`T1078`** is Valid Accounts. It means using a real account to gain or keep access.
- **Level 13:** raised from level 10 because a user gaining higher rights can lead to full control of the system.

❖ **Result:** alerts now carry technique and tactic names, so an analyst sees how the attack works at a glance.

Part B: Proving the Rules with wazuh-logtest

Starting wazuh-logtest and Reading Its Phases

⚙ **Purpose:** **`wazuh-logtest`** lets you paste one log line and see how Wazuh decodes it and which rule matches. No real attack is needed.

⌨ **Command:** `sudo /var/ossec/bin/wazuh-logtest`

- **`/var/ossec/bin/wazuh-logtest`:** the interactive test tool. After it starts, paste a log line and press Enter.

➤ **Phase 1, pre-decoding:** Wazuh reads the full event as one line.

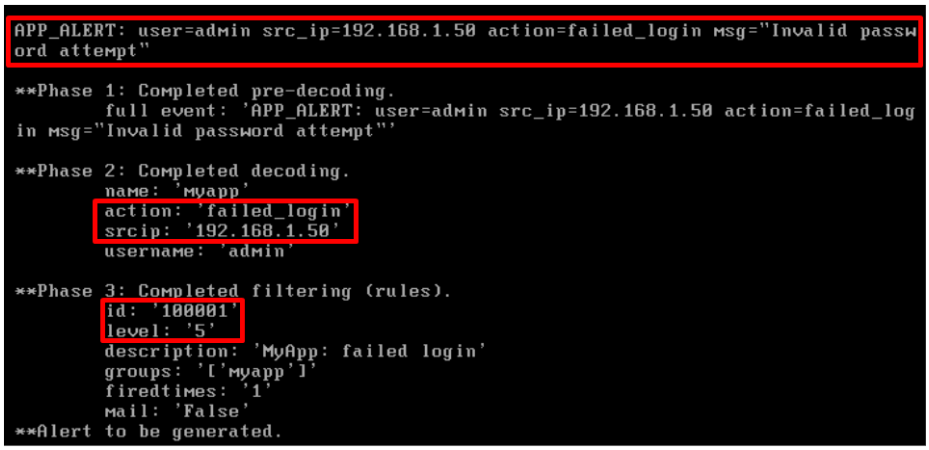
➤ **Phase 2, decoding:** the decoders extract fields such as action, srcip and username.

➤ **Phase 3, filtering:** the rules are checked, and the matching rule id, level and description are shown.

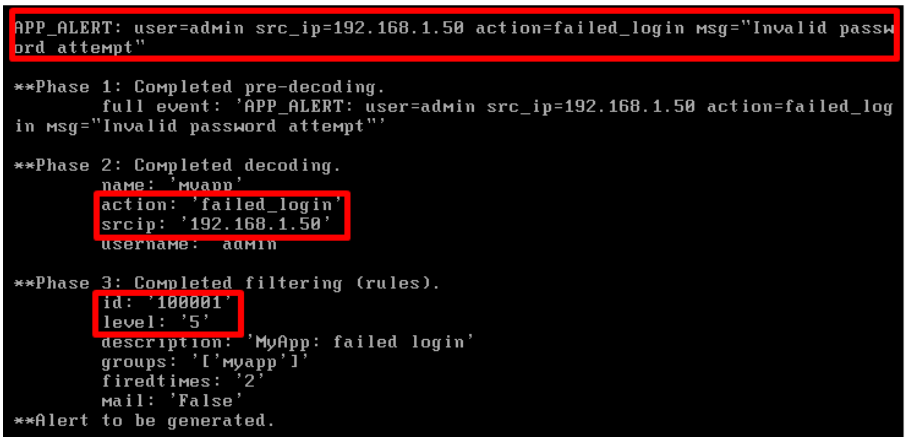
Testing a Failed Login

⚙ **Purpose:** confirm that a failed login line is decoded and matched by rule 100001.

⌨ **Test line:** `APP_ALERT: user=admin src_ip=192.168.1.50 action=failed_login msg="Invalid password attempt"`



Screenshot: wazuh-logtest result for the first failed login



Screenshot: wazuh-logtest result for the second failed login

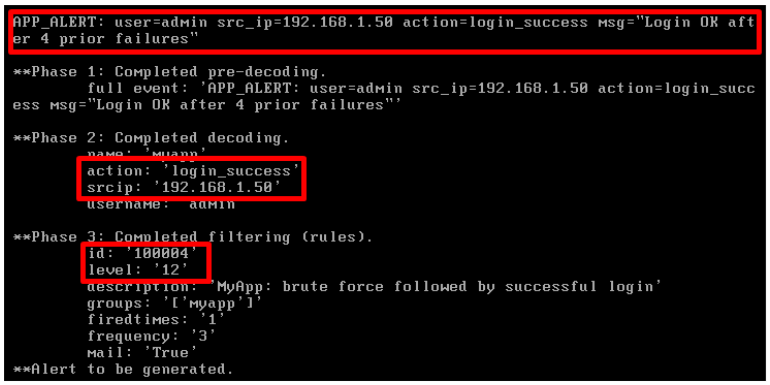
- **Expected output:** the line ends with **Alert to be generated.**
- **Decoding:** name myapp, action failed_login, srcip 192.168.1.50 and username `admin`.
- **Rule match:** id 100001, level 5 and description MyApp: failed login.
- **firedtimes:** counts how many times the rule fired. It is 1 in the first test and 2 in the second. The correlation rule depends on this counting.

❖ **mail False:** level 5 is below the email level, so no email is marked.

Testing Brute Force Followed by Success

⚙ **Purpose:** confirm that a successful login after failed logins triggers rule 100004.

⌨ **Test line:** `APP_ALERT: user=admin src_ip=192.168.1.50 action=login_success msg="Login OK after 4 prior failures"`



Screenshot: wazuh-logtest result for login success after failures

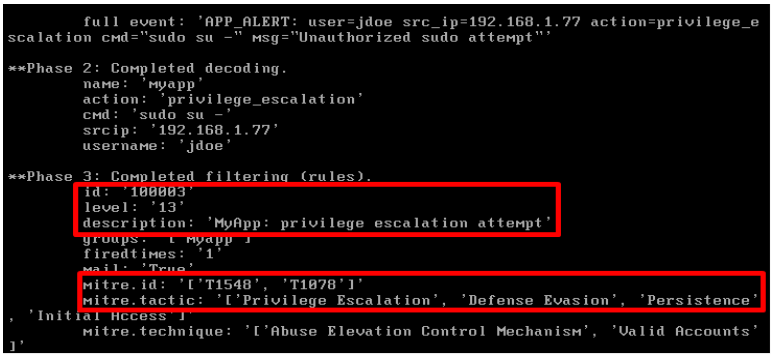
- **Rule match:** id 100004, level 12, description MyApp: brute force followed by successful login and frequency 3.
- **mail True:** with the default settings, level 12 and above is marked for email.

❖ The words inside `msg` are only text. Wazuh counted the real failed login events by itself.

Testing Privilege Escalation

⚙ **Purpose:** confirm rule 100003 and its MITRE ATT&CK labels.

⌨ **Test line:** `APP_ALERT: user=jdoe src_ip=192.168.1.77 action=privilege_escalation cmd="sudo su -" msg="Unauthorized sudo attempt"`



Screenshot: wazuh-logtest result for privilege escalation

- **Decoding:** the **`myapp-cmd`** decoder stored **`sudo su -`** in the `cmd` field. `sudo` runs a command as administrator and `su -` opens a root shell.
- **Rule match:** id `100003`, level `13`, with ids `T1548` and `T1078`.
- **Tactics shown:** Privilege Escalation, Defense Evasion, Persistence and Initial Access.

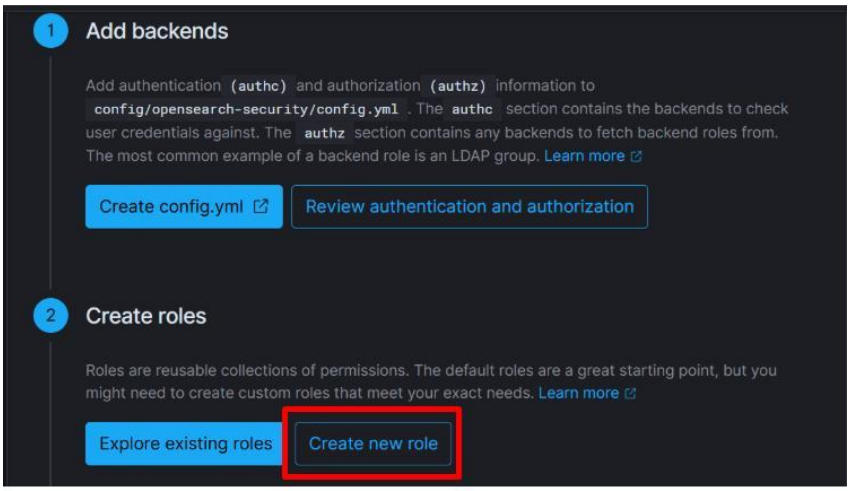
Part C: Role Based Access Control for the SOC Analyst

★ **RBAC** means each person gets only the rights that the job needs.

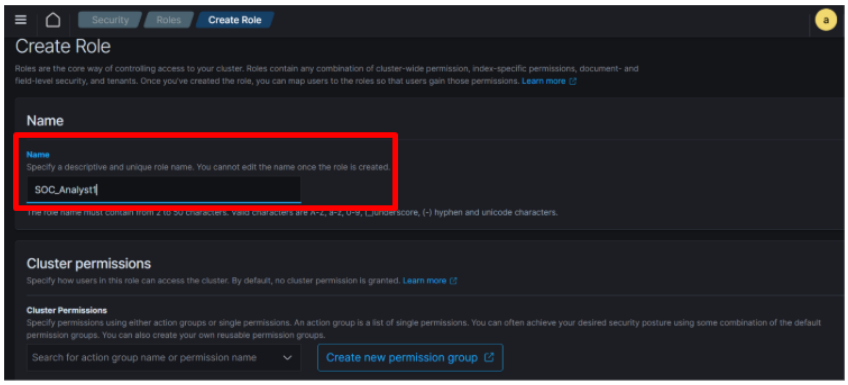
Creating the Analyst Role

⚙ **Purpose:** a Security Operations Center (SOC) analyst must read alerts but must not change settings or delete evidence. A custom role holds exactly those rights.

⌨ **Path:** Security, then Roles, then **Create new role**.



Screenshot: Create new role button

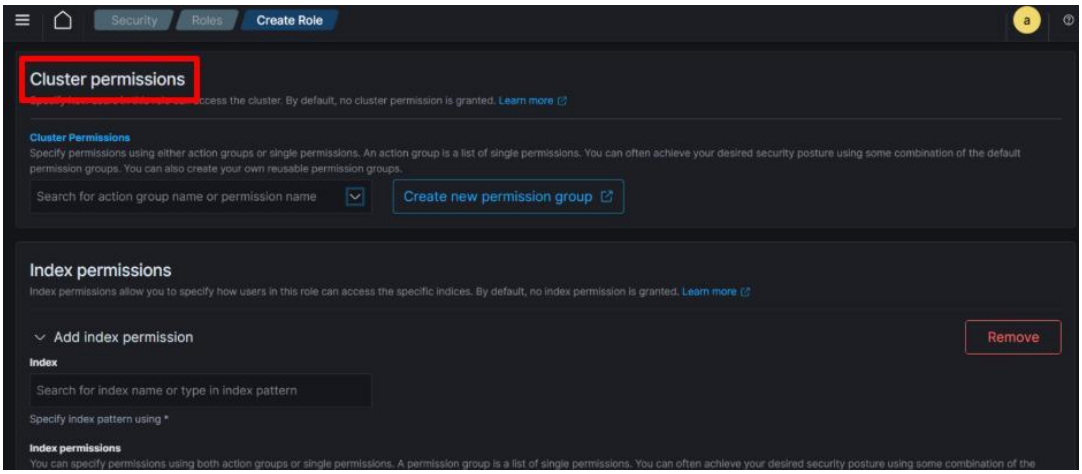


Screenshot: role name SOC_Analyst1

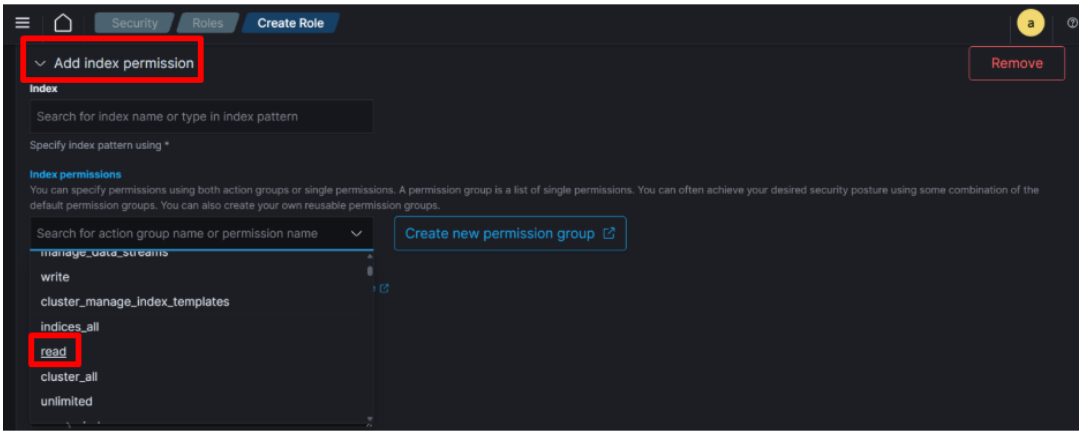
- **Expected output:** the Create Role page opens and the name `SOC_Analyst1` is typed.
- A role is a reusable collection of permissions. The name cannot be changed after the role is created.

Setting Cluster, Index and Tenant Permissions

- **Purpose:** limit what the role can do on the cluster, on the data and in the dashboards.



Screenshot: cluster permissions section

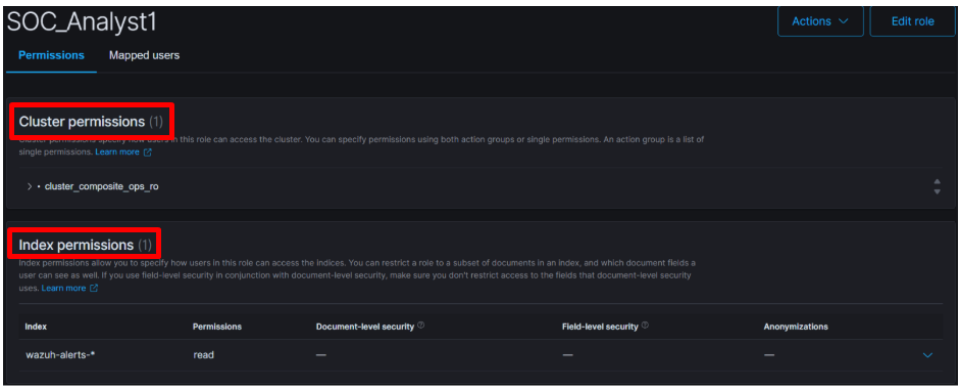


Screenshot: index permission with the read action group

- **Cluster permissions:** control actions on the whole cluster, such as settings. The box was left empty at first so the role stays small.
- **Index pattern `wazuh-alerts-*`:** the indices where Wazuh stores alerts. The star `*` matches every index that starts with this name.
- **Action group `read`:** allows searching and reading documents only. It does not allow writing or deleting.

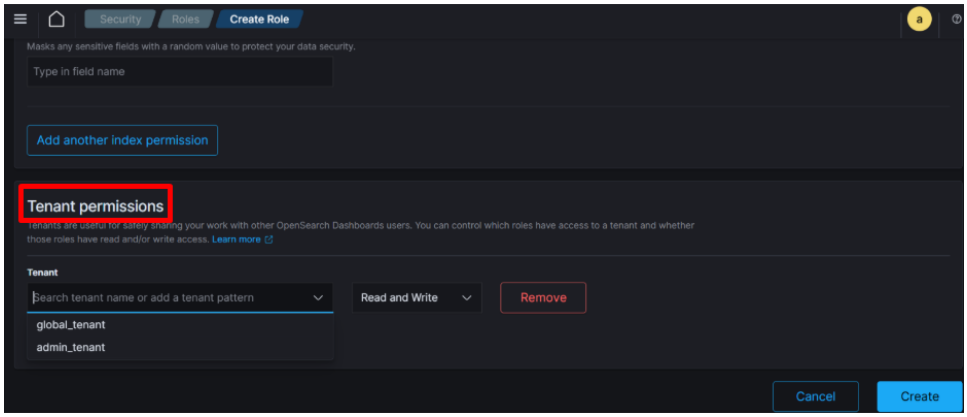
- **Why:** the analyst can study alerts but cannot edit or erase the evidence.

Role Summary and Tenant Permissions

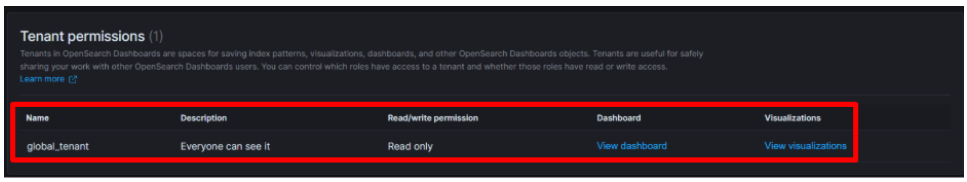


Screenshot: cluster and index permissions in the role summary

- **Expected output:** one cluster permission, **`cluster_composite_ops_ro`,** and one index permission, **`wazuh-alerts-*`** with `read`.
- **`cluster_composite_ops_ro`** is a read-only group that lets dashboards run searches. It cannot change cluster settings.



Screenshot: tenant permissions during role creation



Screenshot: tenant permissions in the role summary

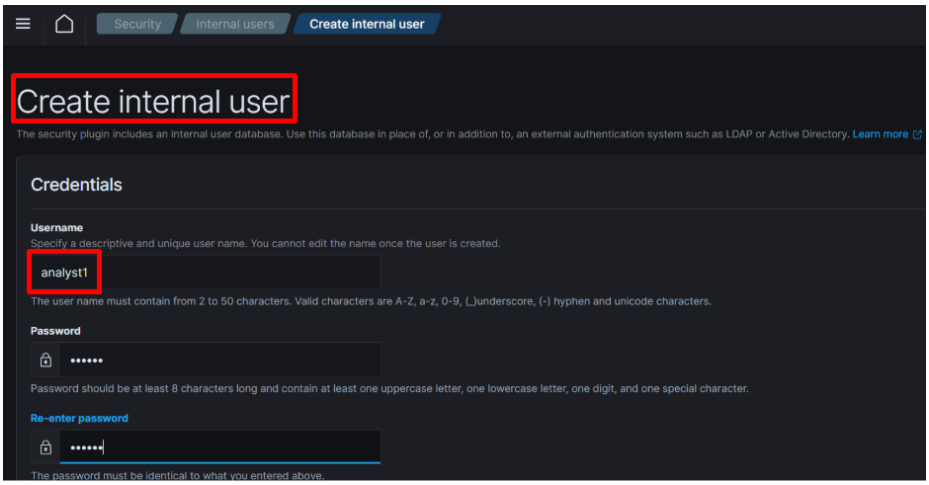
- **Tenants** are spaces in the dashboards where saved searches, visualizations and dashboards live.
- **Result:** the role gets **`global_tenant`** with **read only** permission. The analyst can open shared dashboards but cannot change them.

Creating the Analyst User and Mapping the Role

- **Purpose:** a role does nothing until a user is attached to it. A named account also shows who did what.



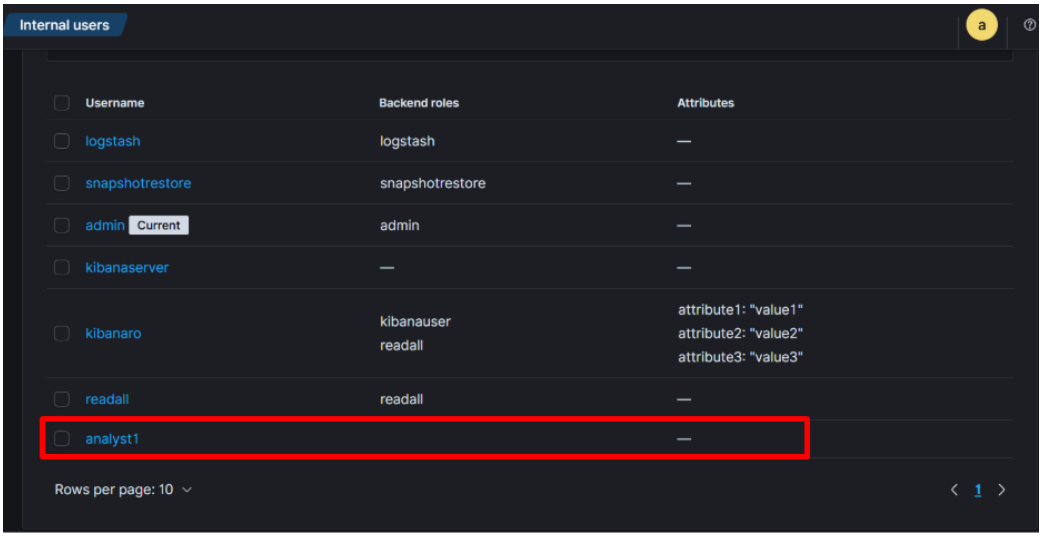
Screenshot: SOC_Analyst1 in the roles list



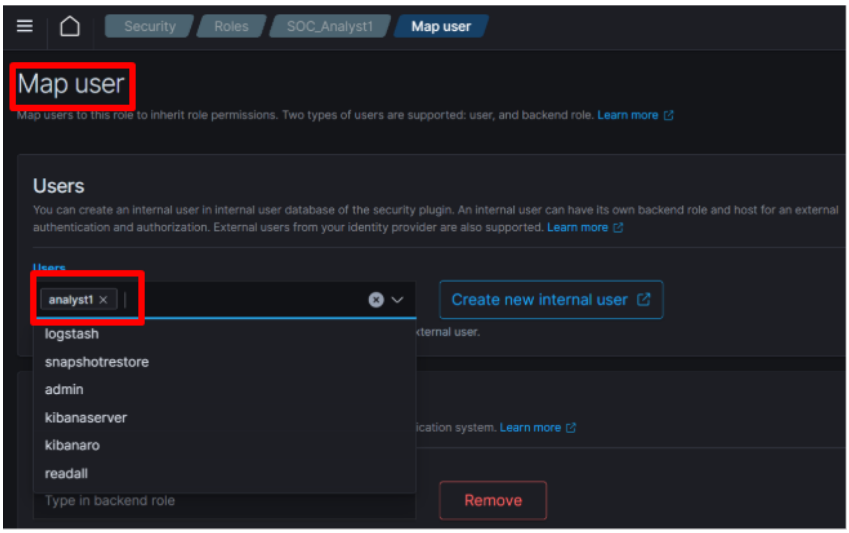
Screenshot: creating the internal user analyst1

- **Roles list:** `SOC_Analyst1` is type **Custom**, with index pattern **`wazuh-alerts-*`** and tenant **`global_tenant`.**
- **Create internal user:** the user name is `analyst1`. The password must have at least 8 characters with an uppercase letter, a lowercase letter, a digit and a special character.

User List and Role Mapping



Screenshot: analyst1 in the internal users list



Screenshot: mapping analyst1 to the role

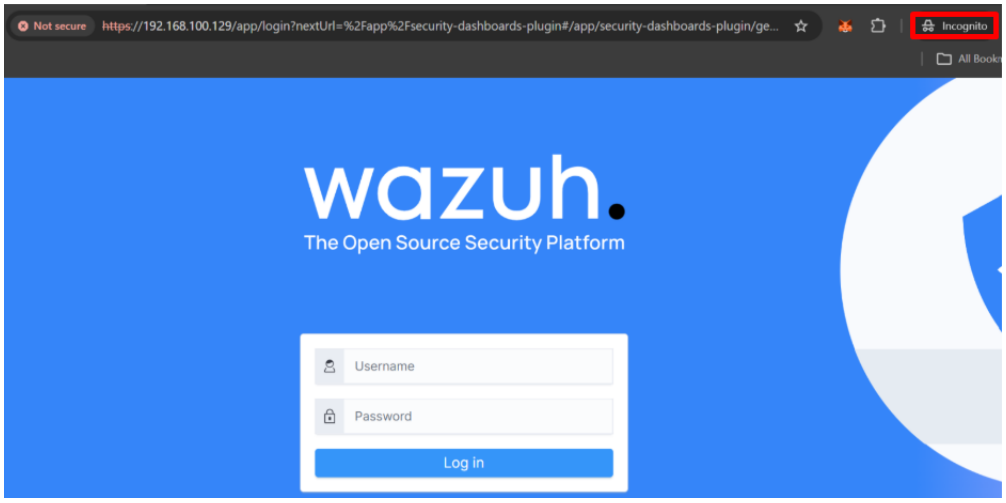
- **Expected output:** `analyst1` appears next to the default users, and the Map user page shows `analyst1` selected.
- **Mapping** connects the user to `SOC_Analyst1`, so the user inherits its permissions.

❖ **Principle of least privilege:** the analyst gets only what is needed. One account per analyst also gives clear accountability in the logs.

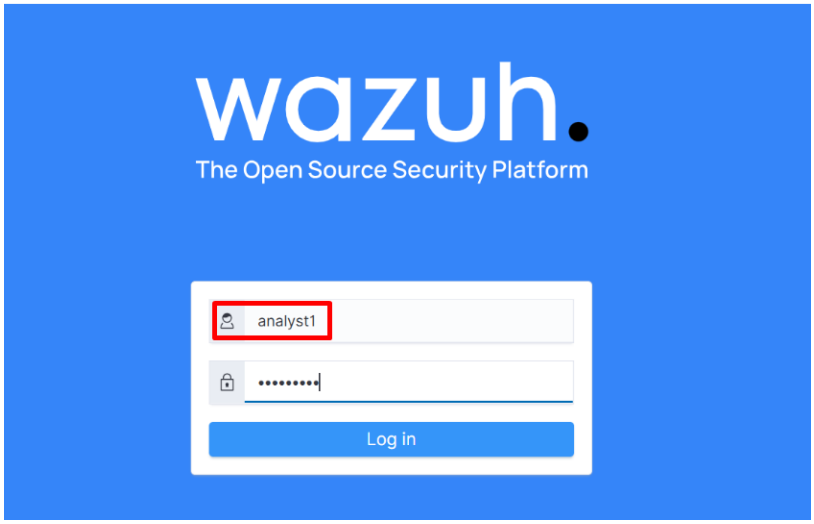
Part D: Verifying Analyst Access and Alerts

Logging In as the Analyst

- **Purpose:** test the role from the analyst point of view. An **incognito** window keeps a separate session, so the administrator session stays untouched.



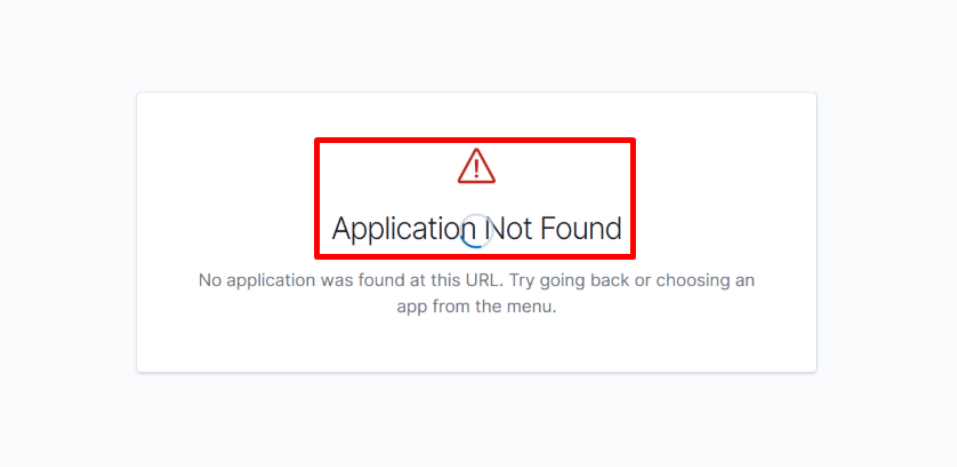
Screenshot: Wazuh login page in an incognito window



Screenshot: logging in as analyst1

- **Expected output:** the Wazuh login page shows the Incognito label, and `analyst1` is typed in the user name box.

Checking the Access Limit After Login

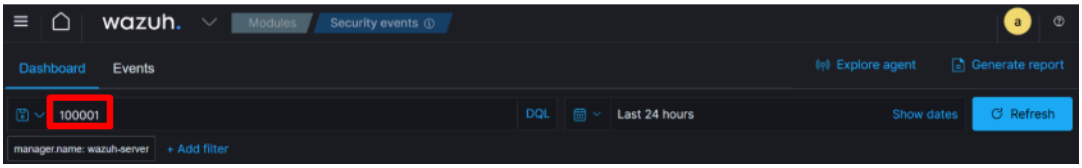


Screenshot: Application Not Found message

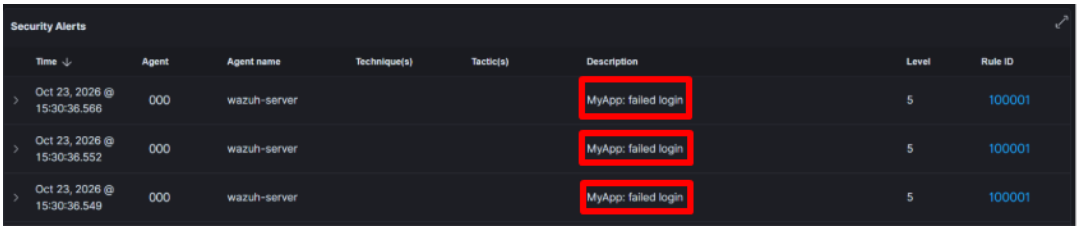
- The login link points to the Security management page. The analyst has no right to manage security, so the dashboard shows **Application Not Found**.
- **This is the expected result.** The role keeps the analyst away from administration pages.

Searching the Alerts

⚙ **Purpose:** confirm that the rules created real, searchable alerts.



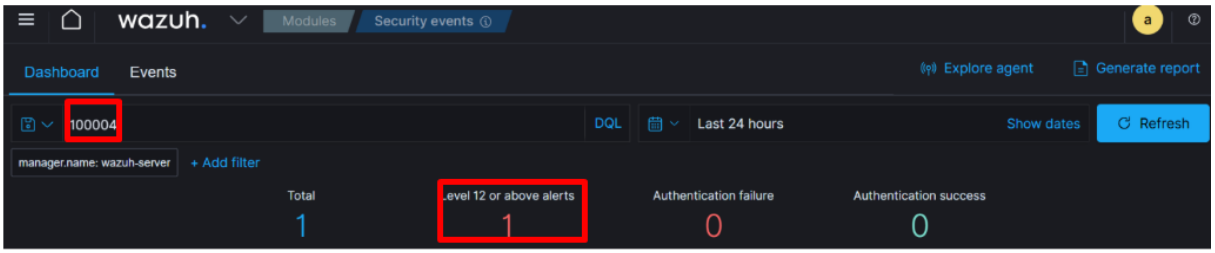
Screenshot: search for rule 100001



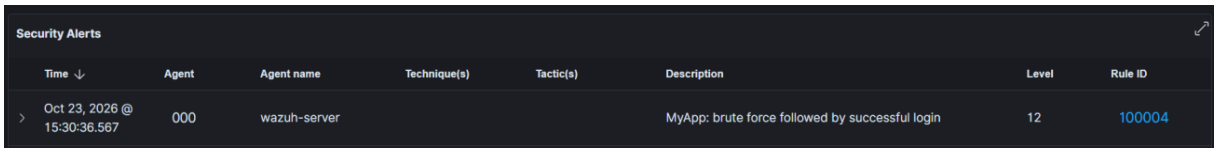
Screenshot: failed login alerts for rule 100001

- Typing `100001` in the search bar lists three **MyApp: failed login** alerts at level 5 from agent `wazuh-server`.

Brute Force Alert and Combined Search

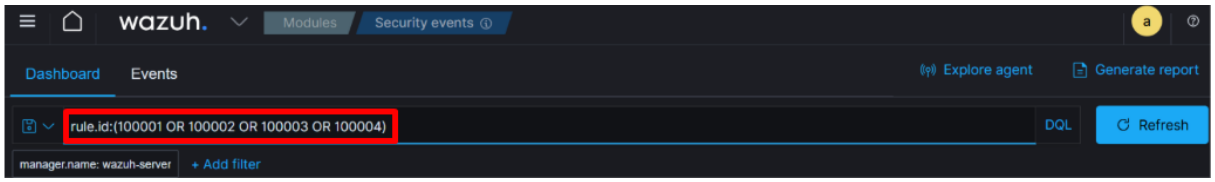


Screenshot: search for rule 100004 with the alert counters

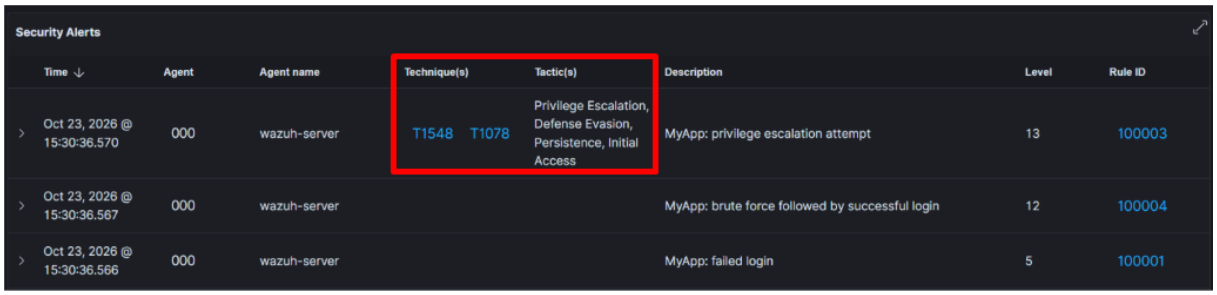


- **Screenshot: brute force alert for rule 100004**

- **Rule 100004:** the counters show Total 1 and Level 12 or above alerts 1. The authentication counters stay 0 because these alerts belong to the custom group `myapp`.



Screenshot: search for all four custom rules



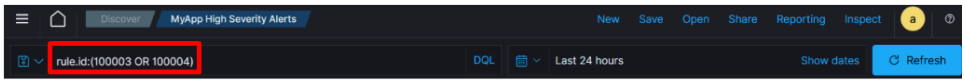
Screenshot: alerts with MITRE techniques and tactics

- **Combined search:** `rule.id:(100001 OR 100002 OR 100003 OR 100004)` uses the Dashboards Query Language (DQL).
- **Result:** rule 100003 shows techniques `T1548` and `T1078` with its tactics, rule 100004 shows level 12, and rule 100001 shows level 5.

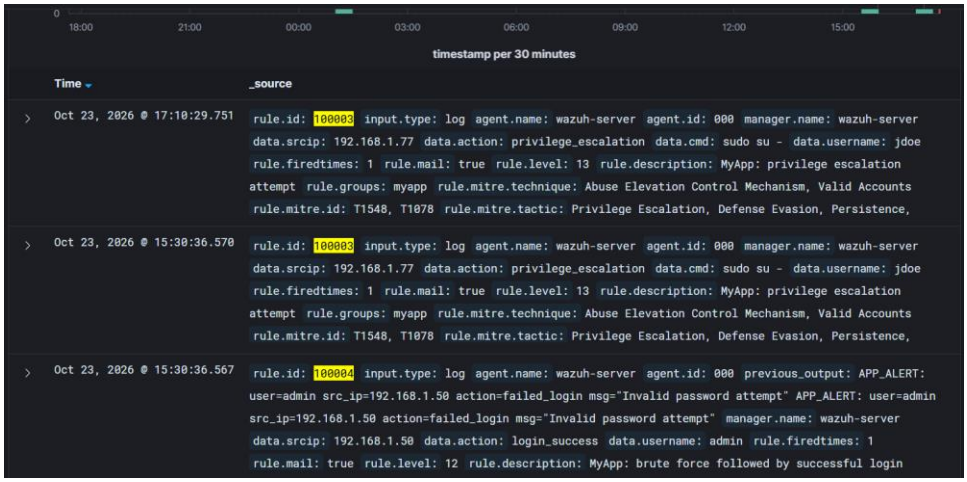
Part E: Dashboard Design

Saving a High Severity Search

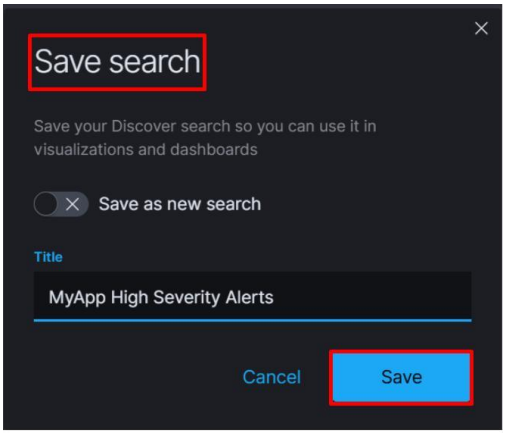
⚙ **Purpose:** a saved search keeps a useful query so that visualizations and dashboards can reuse it.



Screenshot: query for high severity rules



Screenshot: Discover results for rules 100003 and 100004

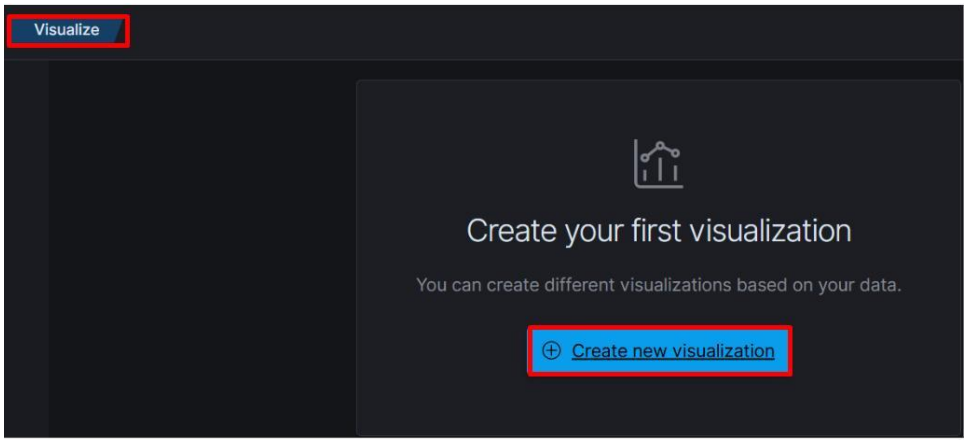


Screenshot: saving the search as MyApp High Severity Alerts

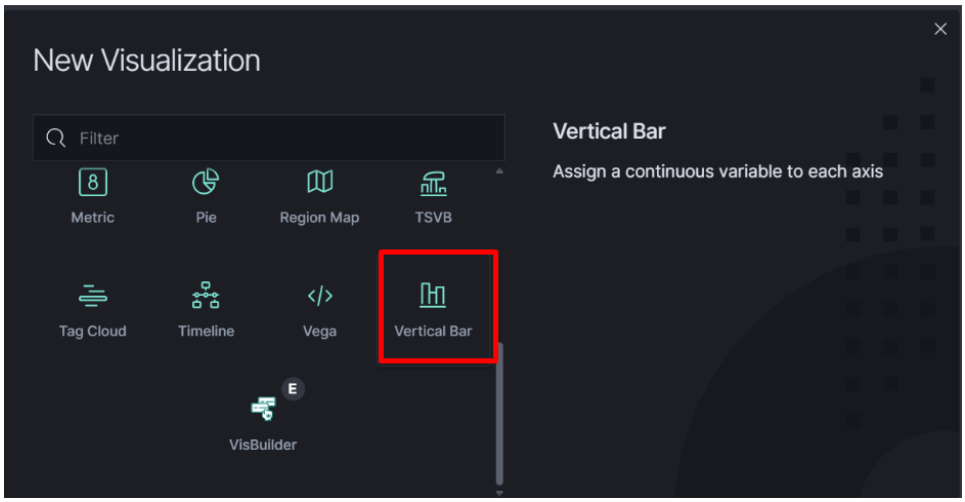
- **Query:** `rule.id:(100003 OR 100004)` keeps only level 12 and level 13 alerts, for the last 24 hours.
- **Results:** two alerts of rule 100003 and one alert of rule 100004. Fields such as `data.srcip`, `data.cmd` and `rule.mitre.id` are visible.
- **Saved as:** `MyApp High Severity Alerts`.

Building the Visualization

- **Purpose:** a chart shows how alerts change over time, which is easier to read than raw rows.



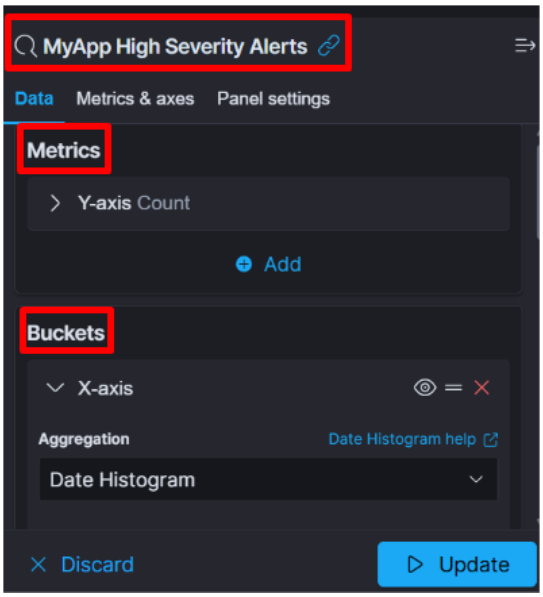
Screenshot: Create new visualization button



Screenshot: Vertical Bar visualization type

- **Path:** Visualize, then **Create new visualization**, then **Vertical Bar**.
- A vertical bar chart is a good fit because it counts alerts in each time slot.
- The chart is linked to the saved search **`MyApp High Severity Alerts`**, so it shows only high severity alerts.

Chart Settings and Result



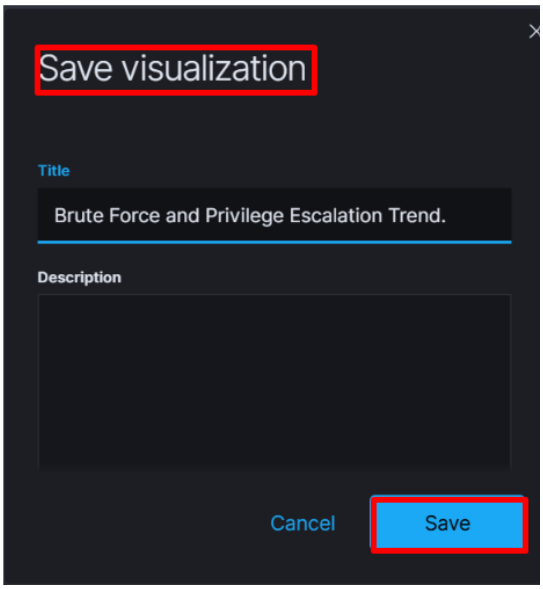
Screenshot: metrics and buckets settings



Screenshot: bar chart of high severity alerts

- **Metrics:** the Y-axis uses **Count**, which is the number of alerts.
- **Buckets:** the X-axis uses a **Date Histogram**, which groups alerts by time.
- **Result:** the chart for the last 48 hours shows bars at about 01:00, 15:00 and 17:00. The legend lists the alert times, so each coloured block is one alert.

Saving the Visualization

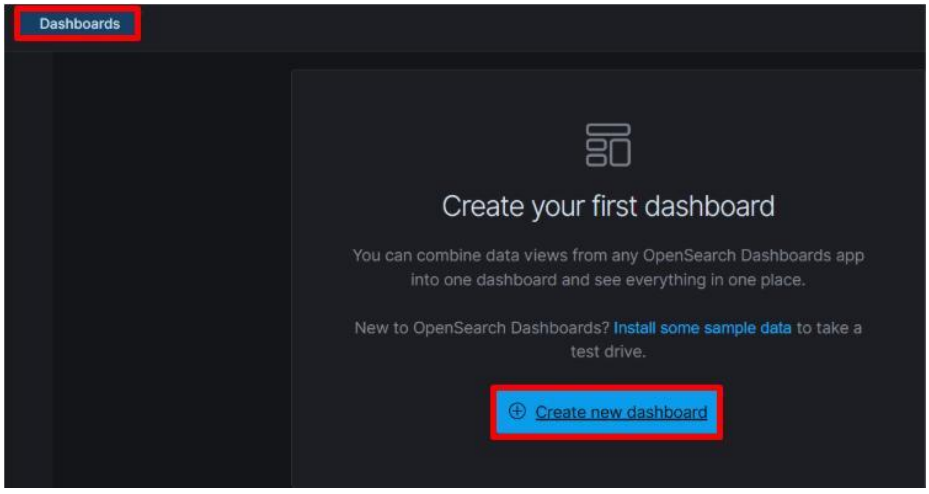


Screenshot: saving the visualization

- **Saved as:** `Brute Force and Privilege Escalation Trend.` so it can be added to a dashboard.

Creating the Dashboard

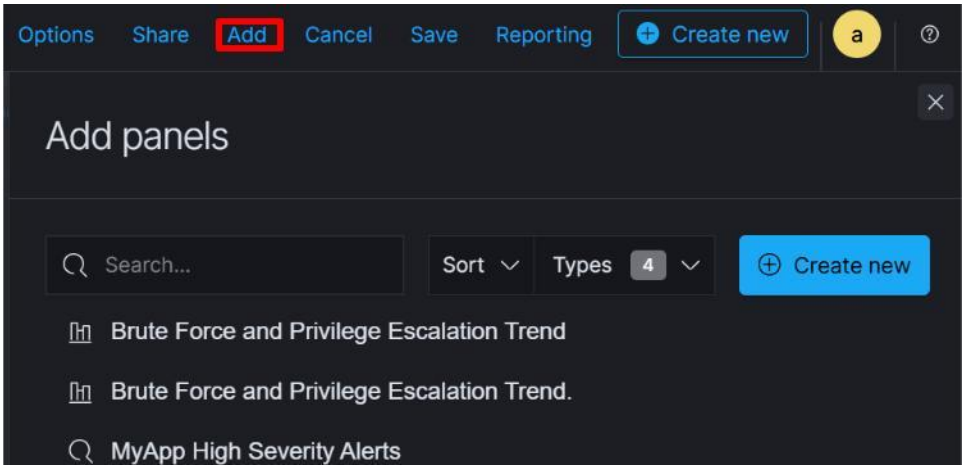
- **Purpose:** a dashboard puts the chart and the alert list on one screen.



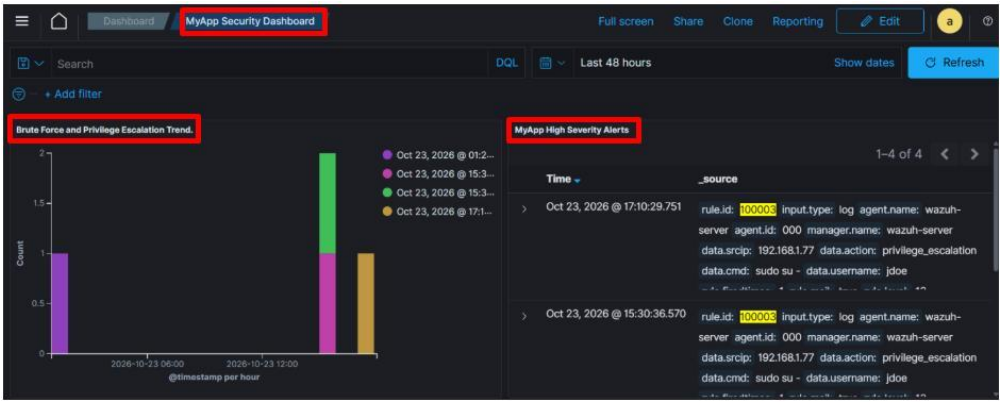
Screenshot: Create new dashboard button

- **Path:** Dashboards, then **Create new dashboard**.

Adding Panels and Final Dashboard



Screenshot: adding panels to the dashboard



Screenshot: MyApp Security Dashboard

- **Add panels:** the Add button opens a list of saved items. The visualization and the saved search are added as panels.
- **Result: MyApp Security Dashboard** shows the trend chart on the left and the high severity alert list on the right, for the last 48 hours.

Conclusion

- Wazuh now reads the application log, decodes it and raises alerts at level 5, level 12 and level 13.
- Rule 100004 catches brute force followed by a successful login, and rule 100003 flags privilege escalation with T1548 and T1078.
- **`wazuh-logtest`** proved each rule, and the dashboard showed the alerts in real data.
- The read-only analyst role follows least privilege.
- **Key lessons:** test the configuration before a restart, and join related events to find attacks that single events hide.